Why not page numbers

The messages endpoint can return thousands of messages for a busy channel, so it has to return them in pages. Page numbers with an offset work for small data, but the database must skip all the earlier rows on every request, and the later pages get slower and slower.

Cursor pagination

I use cursor pagination. The first request asks for the newest messages. The response contains the page of messages and a cursor that marks the last message the client received. To get the next page, the client sends that cursor back, and the server returns the messages that come after the marked one. The server does not count rows to skip. It asks the database for the rows after a position, which an index can answer directly, so every page costs about the same however deep the client has gone.

Sort key

Messages are sorted by the pair (created_at, id), newest first. The timestamp alone is not enough, because two messages can be created in the same millisecond and then their relative order would be undefined. The id is unique and never changes, so adding it as a second key gives every message exactly one place in the sort. The cursor refers to a position in that sort, so a message never appears on two pages and never gets skipped when new messages arrive at the top.

Cursor format

The exact cursor format is on my whiteboard, please see whiteboard-scan.pdf.

Page size

The client can pass a limit parameter to say how many messages it wants in one page, and the default is 25.